



2024

程序插桩和变异测试

目录

CONTENTS

PART 01 程序插桩概述

PART 02 变异测试概述

变异测试定义

应用场景

基本假设

PART 03 变异测试方法

变异定义

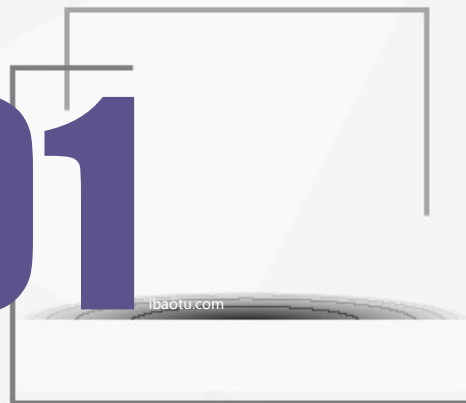
变异算子

变异测试过程

变异得分

变异测试示例

01



程序插桩概述



程序插桩概述



程序插桩是一种基本的软件分析手段，可以帮助研发人员有效获取程序的状态信息，在软件动态测试中具有广泛的应用



理论上，程序中可插桩位置和可插桩语句的数量是不受限制的。然而，程序插桩是需要成本的，不合理的插桩结果会降低程序的运行效率，也增加了研发人员的负担，因此在程序插桩前需要合理选择程序插桩的位置和内容



程序插桩概述



一般的，在程序插桩时应当考虑以下四个问题：

- (1) 需要获取的信息是什么？
- (2) 程序插桩的位置在哪里？
- (3) 程序插桩的数目是多少？
- (4) 插桩语句的类型是什么？



在解答完上述问题后即可开展程序插桩操作。在程序插桩时，应利用尽可能少的插桩点完成尽量多的信息收集工作



程序插桩概述

图 4-1给出了一个求最大公约数的示例程序P1

对P1进行测试时，可通过插桩记录每条语句或语句块的运行次数来分析语句和分支的覆盖情况

因此，可在每个语句块的首条可运行语句前和每条分支语句前后植入计数变量ci和自增语句ci++

s0	int gsd (int x, int y) {
s1	int q = x;
s2	int r = y;
s3	while (q != r) {
s4	if (q > r)
s5	q = q - r;
s6	else
s7	r = r - q;
s8	}
s9	return q;
s10	}

图 4-1 一个示例程序P1

s0	int gsd (int x, int y) {
s1	int c1, c2, c3, c4, c5, c6;
s2	c1++;
s3	int q = x;
s4	int r = y;
s5	c2++;
s6	while (q != r) {
s7	c3++;
s8	if (q > r) {
s9	c4++;
s10	q = q - r;
s11	} else {
s12	c5++;
s13	r = r - q;
s14	} }
s15	c6++;
s16	print(c1, c2, c3, c4, c5, c6);
s17	return q;
s18	}

图 4-2 示例程序P1的插桩程序



程序插桩概述



要实现程序覆盖信息的获取，最直接的方式就是在每条字节码前注入一个探针。然而，该方法是十分低效的。因此，需要设计更有效地字节码插桩规则



插桩规则包括以下四种：

- 顺序插桩
- 无条件跳转插桩
- 有条件跳转插桩
- 返回插桩



程序插桩概述



顺序插桩

对于顺序结构，在语句段内注入一条探针即可。如图 4-6，顺序语句段包含 A 和 B 两条语句。此时，在两条语句间注入一条探针 P 即可

ibaidu.com

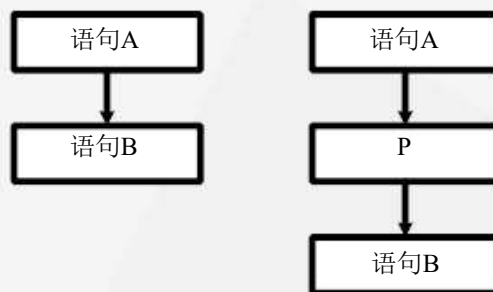


图 4-6 顺序插桩示例



程序插桩概述



无条件跳转插桩

对于无条件跳转结构，与顺序结构相同，其所包含的语句必定会运行，因此在此在语句段前注入一条探针即可。如图 4-7，无条件跳转语句段包含**GOTO** 和

ibaidu.com

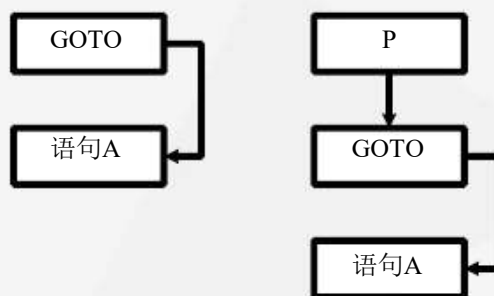


图 4-7 无条件跳转插桩示例



程序插桩概述



有条件跳转插桩

对于有条件跳转结构，程序在运行时只会走某一侧，因此需要在两侧都注入探针。如图 4-8，有条件跳转语句段包含A、B、C 等三条语句。此时，需要

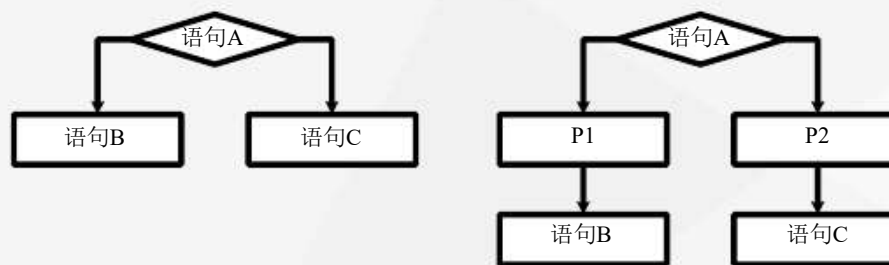


图 4-8 有条件跳转插桩示例



程序插桩概述



返回插桩

对于返回结构，在返回语句前注入一个探针即可。如图 4-9，返回语句段包含RETURN 一条语句。此时，在RETURN 前注入一条探针即可

ibaozu.com

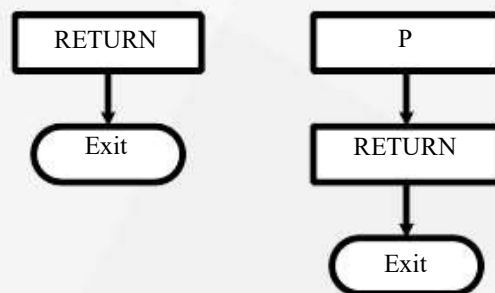


图 4-9 返回插桩示例



02

变异测试概述

目录

CONTENTS

PART 01 程序插桩概述

PART 02 变异测试概述

变异测试定义

应用场景

基本假设

PART 03 变异测试方法

变异定义

变异算子

变异测试过程

变异得分

变异测试示例



变异测试定义



变异测试也称为“变异分析”，是一种对测试数据集的有效性、充分性进行评估的技术，能为研发人员开展需求设计、单元测试、集成测试提供有效的帮助



变异测试通过对比源程序与变异程序在执行同一测试用例时差异来评价测试用例集的错误检测能力



在变异测试过程中，一般利用与源程序差异极小的简单变异体来模拟程序中可能存在的各种缺陷

目录

CONTENTS

PART 01 程序插桩概述

PART 02 变异测试概述

变异测试定义

应用场景

基本假设



PART 03 变异测试方法

变异定义

变异算子

变异测试过程

变异得分

变异测试示例



应用场景



在软件测试时，若当前测试用例未能检测到软件缺陷，则存在两种情形：

- ① 软件已满足预设的需求，软件质量较高
- ② 当前测试用例设计不够充分，不能有效检测到软件中的缺陷



逻辑测试和路径测试方法，分别从程序实体覆盖和路径覆盖的角度来评估软件测试的充分性。然而，这些方法并不能直观地反映测试用例的缺陷检测能力

变异测试方法可用于度量测试用例缺陷检测能力

目录

CONTENTS

PART 01 程序插桩概述

PART 02 变异测试概述

变异测试定义

应用场景

基本假设



PART 03 变异测试方法

变异定义

变异算子

变异测试过程

变异得分

变异测试示例

基本假设



变异测试的可行性主要基于两点假设：

- 熟练程序员假设，关注于熟练程序员的编程行为
- 变异耦合效应假设，关注于变异程序的缺陷类型



熟练程序员假设是指熟练程序员由于开发经验丰富、编程水平较高，其编写的代码即使包含缺陷，也与正确代码十分接近。此时，针对缺陷代码仅需做微小的修改即可使代码恢复正确



变异耦合效应假设是指若测试用例能够杀死简单变异体，那么该测试用例也易于杀死复杂变异体

03

ibaotu.com

变异测试方法

目录

CONTENTS

PART 01 程序插桩概述

PART 02 变异测试概述

变异测试定义

应用场景

基本假设

PART 03 变异测试方法

变异定义

变异算子

变异测试过程

变异得分

变异测试示例



变异定义



程序变异指基于预先定义的变异操作对程序进行修改，进而得到源程序变异程序（也称为变异体）的过程

- 当源程序与变异程序存在执行差异时，则认为该测试用例检测到变异程序中的错误，变异程序被杀死
- 当两个程序不存在执行差异时，则认为该测试用例没有检测到变异程序中的错误，变异程序存活

目录

CONTENTS

PART 01 程序插桩概述

PART 02 变异测试概述

变异测试定义

应用场景

基本假设

PART 03 变异测试方法

变异定义

变异算子

变异测试过程

变异得分

变异测试示例





变异算子

表 4-1 面向过程程序的变异算子

变异算子	描述
运算符变异	对关系运算符 “<” 、 “<=” 、 “>” 、 “>=” 进行替换，如将 “<” 替换为 “<=”
	对自增运算符 “++” 或自减运算符 “--” 进行替换，如将 “++” 替换为 “--”
	对与数值运算的二元算术运算符进行替换，如将 “+” 替换为 “-”
	将程序中的条件运算符替换为相反运算符，如将 “==” 替换为 “!=”
数值变异	对程序中整数类型、浮点数类型的变量取相反数，如将 “i” 替换为 “-i”
方法返回值变异	删除程序中返回值类型为void的方法
	对程序中方法的返回值进行修改，如将 “true” 修改为 “false”

变异算子

表 4-2 面向对象程序的变异算子

变异算子	描述
继承变异	增加或删除子类中的重写变量
	增加、修改或重命名子类中的重写方法
	删除子类中的关键字 <code>super</code> ，如将 <code>"return a*super.b"</code> 修改为 <code>"return a*b"</code>
多态变异	将变量实例化为子类型
	将变量声明、形参类型改为父类型，如将 <code>"Integer i"</code> 修改为 <code>"Object i"</code>
	赋值时将使用变量替换为其它可用类型
重载变异	修改重载方法的内容，或删除重载方法
	修改方法参数的顺序或数量

目录

CONTENTS

PART 01 程序插桩概述

PART 02 变异测试概述

变异测试定义

应用场景

基本假设

PART 03 变异测试方法

变异定义

变异算子

变异测试过程

变异得分

变异测试示例





变异测试过程



在变异测试过程中，程序与变异程序的执行差异主要表现为以下两个情形：

- (1) 执行同一测试用例时，源程序和变异程序产生了不同的运行时状态
- (2) 执行同一测试用例时，源程序和变异程序产生了不同的执行结果



根据满足执行差异要求的不同，可将变异测试分为弱变异测试 (Weak Mutation Testing) 和强变异测试 (Strong Mutation Testing)



在弱变异测试过程中，当情形 (1) 出现时就可认为变异程序被杀死，而在强变异测试过程中，只有情形 (1) 和 (2) 同时满足才可认为变异程序被杀死



变异测试过程

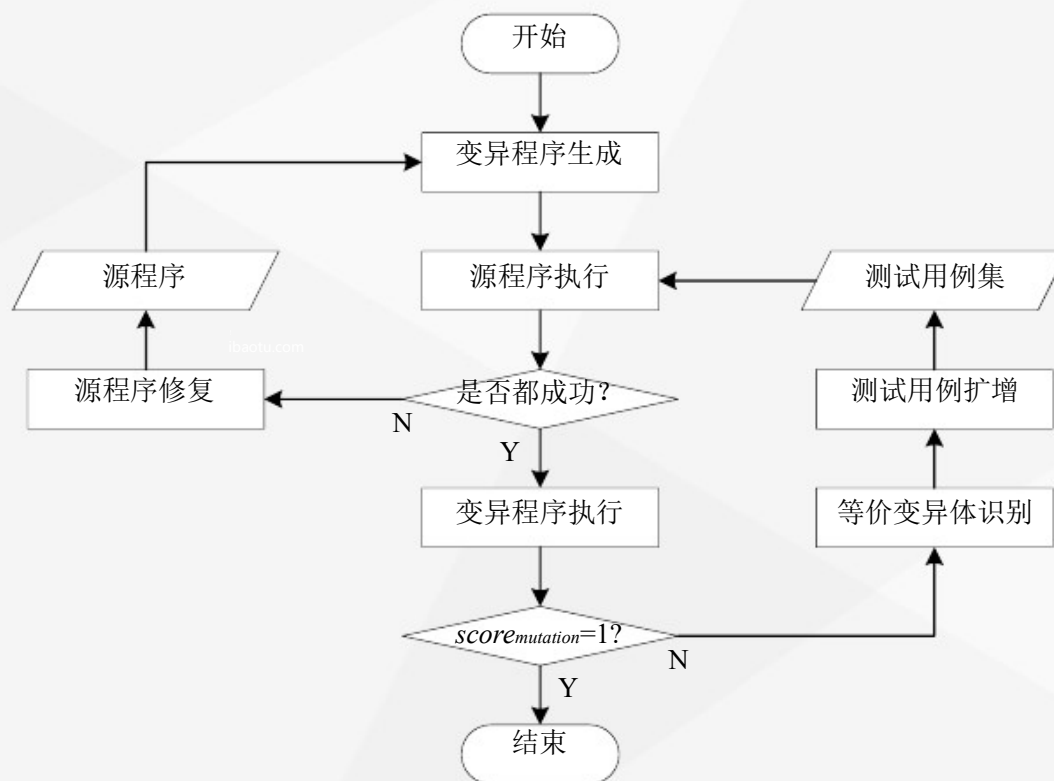


图 4-12 变异测试基本流程图

目录

CONTENTS

PART 01 程序插桩概述

PART 02 变异测试概述

变异测试定义

应用场景

基本假设

PART 03 变异测试方法

变异定义

变异算子

变异测试过程

变异得分

变异测试示例



变异得分



变异得分 (Mutation Score) 是一种评价测试用例集错误检测有效性的度量指标



变异得分 $score_{mutation}$ 的值介于 0 与 1 之间, 数值越高, 表明被杀死的变异程序越多, 测试用例集的错误检测能力越强, 反之则越低

- 当 $score_{mutation}$ 的值为 0 时, 表明测试用例集没有杀死任何一个变异程序
- 当 $score_{mutation}$ 的值为 1 时, 表明测试用例集杀死了所有非等价的变异程序

$$score_{mutation} = \frac{num_{killed}}{num_{total} - num_{equivalent}}$$

目录

CONTENTS

PART 01 程序插桩概述

PART 02 变异测试概述

变异测试定义

应用场景

基本假设

PART 03 变异测试方法

变异定义

变异算子

变异测试过程

变异得分

变异测试示例





变异测试示例

s0	int fun(int x) {
s1	if (x >= 60)
s2	return 1;
s3	else
s4	return 0;
s5	}

mutation



ibaozu.com

测试用例		x	变异杀死结果			
			M1	M2	M3	M4
初始测试用例	t1	80	√		√	
	t2	40	√			√
新增测试用例	t3	60	√	√	√	

s0	int fun(int x) {
s1	if (x < 60)
s2	return 1;
s3	else
s4	return 0;
s5	}

(1) 变异程序M1

s0	int fun(int x) {
s1	if (x > 60)
s2	return 0;
s3	else
s4	return 0;
s5	}

(3) 变异程序M3

s0	int fun(int x) {
s1	if (x > 60)
s2	return 1;
s3	else
s4	return 0;
s5	}

(2) 变异程序M2

s0	int fun(int x) {
s1	if (x > 60)
s2	return 1;
s3	else
s4	return 1;
s5	}

(4) 变异程序M4

谢谢欣赏